

Schist

A Property-Aware Mathematical Language over Lean

Scoped refinements, behavioral contracts,
and proof-carrying elaboration

Central proposal. Extend the type system that authors interact with, not initially the conversion rules of the trusted kernel. Mathematical objects retain their identities while acquiring scoped, proof-backed properties. The next step of an argument determines which properties must be established, and every successful elaboration emits ordinary Lean evidence.

Prepared for Vladimir Reshetnikov

GPT-6 Astra Pro

6 September 2026

This is a design article with mathematical arguments, an unexecuted Lean reference core, and an executed finite-arithmetic companion. It is not an implemented or benchmarked proof language, a fresh verification of either repository, or a mechanically verified compiler. Source status and validation limits are stated in Sections [2](#) and [16](#).

Abstract

A mathematician does not repeatedly rediscover that a previously introduced measure is a probability measure, that a function is differentiable on the region under discussion, or that a normalized counterexample has a prime exponent. These properties form a persistent local interface to the objects of the proof. Lean can already express all these assertions. The problem is that their use is split among bundled structures, ordinary hypotheses, typeclass instances, coercions, theorem search, and tactic-specific side-condition solvers.

SCHIST proposes a conservative, property-aware elaboration layer that makes this interface explicit and usable. It combines open refinement rows on existing objects, relational contracts involving several objects, locality-indexed observations, and theorem-backed backward propagation of requirements. Pure functional behavior, mathematical regularity, and operational effects receive precise but distinct contracts. A certified computation returns evidence for the consumer's requested relation, while Leant supplies candidate terms, structural adapters, and behavior-guided search without acquiring authority over the statement being proved.

The proposal builds on both second-round syntheses rather than replacing their acceptance boundary. Its additional contribution is an account of how the properties needed at that boundary become ordinary, compositional typing obligations. Concrete studies use ProveIt's local derivative theorem and affine independent-copy uniqueness proof, and the FLT repository's Frey package and already concise terminal contradiction. A general orbital-comparison lemma illustrates the resulting mathematical interface. The article specifies a small core calculus, its translation and relative soundness argument, a bounded elaboration strategy, behavioral synthesis contracts, adversarial examples, and an implementation and evaluation plan that separates the value of libraries, services, property typing, and narrative syntax.

Contents

1	The third-round decision	3
2	Sources, inherited conclusions, and evidentiary limits	3
3	What the inspected Lean code actually teaches	5
4	A small author-facing language	6
5	Four layers of mathematical typing	7
6	A conservative core calculus	9
7	Backward propagation of mathematical requirements	11
8	Behavior belongs in types, but not in one undifferentiated annotation	12
9	Worked development: probability-law uniqueness without moment hypotheses	14
10	Locality, induction motives, and mathematical proof plans	16
11	The Frey-package case: mathematical structure without instance ambiguity	17
12	Certified algebra as a client of the type layer	18
13	Leant as a supplier of evidence candidates	20
14	Which type-system extension should actually be implemented?	22
15	Soundness, identity preservation, and their limits	23
16	A concrete implementation slice and reference core	24
17	Publication, maintenance, and semantic visibility	26
18	Evaluation that can reject the proposal	27
19	Answers to the next-round questions	29
20	Conclusion	30
A	Supplementary artifacts and validation	30
B	Source identities and inspection scope	30

1 The third-round decision

1.1 What should change, and what should not

The strongest direction for a third iteration is not another spelling of “calculate, then check.” Both second-round syntheses have already established the right boundary for that operation: a fixed request, a returned value, and a proof of the precise relation that the request demands. They also agree that scoped guards, original-object identity, justified execution, exact-target reconstruction, retained evidence, and an equally equipped Lean baseline are non-negotiable [1, 2].

The remaining authoring problem is what happens *before* such a request. Why is a particular denominator known to be nonzero? Which local identity may be differentiated? What makes the measure in an observation a probability measure? Which properties of a constructed witness are available to the next step? Why does a synthesized function satisfy the intended behavior rather than merely its bare arrow type?

SCHIST answers: *make these properties part of the mathematical typing interface, and elaborate their uses into ordinary proofs.* An object need not be replaced by a new nominal wrapper whenever another useful fact about it is proved. Its local interface can grow monotonically within its scope. A method advertises the facts it needs, so the elaborator can construct and check the appropriate evidence before applying it.

This is an extension of the *surface and elaboration type system*. It is not a claim that Lean lacks dependent types, that behavioral specifications cannot already be written in Lean, or that the kernel needs an SMT solver. Those would misdiagnose the problem. The intended gain is a disciplined integration of capabilities that are currently spread across several mechanisms.

1.2 Five commitments

1. **Retain the object.** Learning a property of x must not silently replace x , change its ambient structure, or strengthen the theorem’s hypotheses.
2. **Make every property evidence-bearing.** A refinement is a Lean proposition with a proof obligation, not an unchecked adjective.
3. **Plan from the demand.** Infer the premises needed for the author’s chosen next step, rather than saturating the context with every conceivable fact.
4. **Keep different guarantees different.** Global equality, local agreement, finite observation, behavioral correctness, and runtime safety are not interchangeable forms of a generic success flag.
5. **Export ordinary mathematics.** Successful work produces ordinary Lean declarations and retained evidence, usable without the new frontend.

The last commitment is not merely an escape hatch. It makes the proposal falsifiable: once all its theorem packages and services are supplied to Lean, the new typing and authoring layers must still demonstrate a benefit.

2 Sources, inherited conclusions, and evidentiary limits

2.1 What was inspected

Both complete main TeX reports were read closely: *Mathematical Objects, Certified Computation* and the revised *Nine Workbenches*. The latter was read at Brainstorm commit b052ec011beb96a7c48df2a9988823f0f779b445. The former’s exact fetched Git blob is recorded in the source register. Their mathematical examples, criticism of the proposals, accepted corrections, implementation recommendations, and questions for the next round inform this article [1, 2].

The code inspection adds ProveIt’s local-derivative and affine-independent-copy modules; the FLT repository’s Frey-package definition, proof-path guide, and terminal contradiction; and current Leant source and documentation for verification, synthesis provenance, and Length behavioral ranking/filtering. Selected introductory sections of Leant’s much larger behavioral-synthesis proposal were also inspected [3, 4, 5, 6, 7, 8, 9, 10, 11].

This is not a new census of either Lean repository, a complete rereading of all nine individual round-two packages, or a reproduction of their experiments. The two syntheses supply the comparative account of those nine proposals. The additional probability file is development material, not a sealed held-out test. Exact file identities and the scope of each inspection appear in Appendix B and `source_manifest.json`.

2.2 What is inherited

The proposed value-plus-specification interface is inherited, as are the separations between exact presentations, observations, and directed abstractions; sufficient guards and weakest conditions; kernel checking and native execution; search failure and negation; and discovery and publication replay. In particular, the corrected second-round examples are treated as regression requirements, not as discoveries of this article [1, 2].

Topic	Round-two foundation	Additional proposal here
Computational meaning	A result proves a requested relation.	Property typing constructs the premises needed to request and use that relation.
Objects and views	Presentations remain tied to an original object.	Open refinements enrich that object’s local interface without changing its carrier.
Guards	Conditions are ordered and scoped.	Backward requirement transformers turn a chosen proof step into a typed, scoped residual problem.
Behavior	A bare function type need not express intended behavior.	Pointwise, relational, higher-order, and operational contracts have distinct elaboration rules.
Automation	Providers are untrusted; exact-target replay is required.	A property-demand graph supplies Leant with bounded composition tasks and exact behavioral obligations.
Evaluation	Compare against the same libraries and services.	Separate the property-typing treatment from the narrative-syntax treatment.

2.3 What the earlier experiments do not establish

The second synthesis’s polynomial experiment checks a Boolean result for a specific coefficient-list checker. It does not yet prove that checker’s interpretation theorem or connect every source expression to its input. Its large-degree family multiplies by a fixed two-term factor; it is not a general multivariate or dense-by-dense performance study. The revised synthesis explicitly acknowledges these limitations [2, toolchain experiments and parallel-synthesis discussion].

Likewise, running many finite arithmetic examples is not a proof of a language’s soundness or usefulness. This article therefore separates four statuses: source-inspected, mathematically argued, executed on finite inputs, and kernel-checked. No new Lean invocation was performed for this article. The supplementary Lean core is complete reference source intended for checking, not a successful compiler receipt. The Python companion’s narrower executed results are reported separately.

3 What the inspected Lean code actually teaches

3.1 Local derivative arguments: the missing interface is not global smoothness

ProveIt’s autonomous-derivative theorem assumes that $s \subseteq \mathbb{R}$ is open, that W has derivative $\phi(W(x))$ at each $x \in s$, and that the family G_n has specified derivatives only at the values $W(x)$ reached from s . It proves

$$W^{(n)}(x) = G_n(W(x)) \quad (n \in \mathbb{N}, x \in s).$$

Its induction step explicitly obtains equality near x from the induction hypothesis on s , transfers derivatives along that eventual equality, and then applies the chain rule [3].

This is mathematically important plumbing, not dispensable decoration. Replacing it by “differentiate both sides” is sound only when the omitted local premises are recovered. Nor is a blanket requirement that all G_n be globally smooth an acceptable simplification: it would strengthen the theorem and defeat the source’s deliberate range-local generality.

A property-aware interface should retain three facts separately:

$$x \in s, \quad s \text{ is open}, \quad \text{EqOn}(s, W^{(n)}, G_n \circ W).$$

A registered theorem combines them into the germ equality needed at x . The derivative method then asks for the derivative certificates at that point. The author’s local mathematical step becomes short because the interface connects these facts, not because the system ignores them.

3.2 Probability: a function type conceals a rich interface

The additional ProveIt file defines an affine independent-copy operator on measures,

$$\mathcal{A}(\mu) = (\rho \text{ map } d) * (\mu \text{ map } (q \cdot -)). \quad (1)$$

Here ρ is the digit law, d is measurable, $q \in \mathbb{R}$, and $*$ denotes convolution. Its uniqueness theorem works with probability measures on a complete, second-countable real inner-product space with its Borel measurable structure. For $|q| < 1$, two probability fixed points of $\mathcal{A}$ are equal. The proof uses characteristic functions, not a finite-moment metric [4].

Several distinct interfaces occur in the source. A measurable pushforward of a probability measure is again a probability measure. Convolution and product-law pushforward are related by a theorem. Characteristic functions are continuous, normalized at zero, and bounded in norm by one. Equality of characteristic functions determines the original measures in the stated setting. None of these facts is contained in the bare types `Measure E` or `E → Complex`. All are expressible as ordinary Lean propositions and many already have library support; the source establishes or installs them explicitly [4].

This is the kind of interface that an open refinement should expose. A method requiring a probability law should recover its existing evidence. It should not search for an unrelated replacement measure, attach an arbitrary new probability structure, or add a moment assumption merely because a preferred uniqueness theorem uses one.

3.3 The Frey package: dependent types already express mathematical properties

The FLT repository already bundles substantial mathematical constraints in `FreyPackage`. The data include integers a, b, c and a natural number p ; proof fields state nonzeroness, primality, $5 \leq p$, the equation $a^p + b^p = c^p$, coprimality, and congruence conditions. Derived lemmas recover positivity, oddness, further coprimality, and nonzeroness of products [5].

This is direct evidence against the claim that Lean’s type system cannot encode mathematical properties. The design issue is whether every use must manually project and rearrange those

properties, and whether every new conjunction requires a separately named structure. Moreover, some constraints concern *several* fields simultaneously. The equation involving a, b, c, p cannot be modeled adequately by independent unary labels on the four components.

The same file defines integral and rational Frey curves separately. Fractions in integral coefficients involve integer division; their rational counterparts involve field division. Identifying the two models requires divisibility and transport arguments, not cosmetic suppression of casts [5].

3.4 Some proofs are already short

The terminal FLT contradiction combines irreducibility, modularity, level lowering, and vanishing of the relevant cusp-form space in a very small proof body. The inspected solution file also contains a large generated instance-control preamble. These are different phenomena [7].

A new language should not claim that the one-line terminal application is a long mathematical argument needing compression. Nor should it count all instance-management text as if each line were a missing human inference. There are at least three costs: mathematical reasoning, elaboration-environment management, and generated-source presentation. They deserve separate remedies and separate measurements.

Design decision. Use local derivatives as a control, affine-law uniqueness as an additional development case, and Frey-package reasoning as a test of relational properties and structure selection. Preserve already concise Lean unchanged.

4 A small author-facing language

4.1 A mathematical binder has data and laws

The following syntax is proposed, not accepted Lean syntax:

```
context D : measurable space
context E : complete second-countable real inner-product space
           with its Borel measurable structure

let rho : Measure D where probability
let digit : D -> E where measurable
let q : Real where abs q < 1

let mu, nu : Measure E where probability
assume mu, nu are fixed points of affine_law rho digit q

show mu = nu
  by characteristic_functions
    compare along the orbit t, q*t, q^2*t, ... tending to 0
```

This is not free-form English to be interpreted afresh on every run. The keywords select registered notation and theorem interfaces. The mathematical expressions between them use Lean’s normal parser and elaborator, with an inspectable translation. For example, `probability` resolves to the property of *this* measure, and `fixed points` resolves to two specified equations involving the same operator.

The context abbreviation expands to explicit class parameters. It is not a license to infer arbitrary completeness or measurability assumptions. The expanded theorem header is available before proof search begins and is part of the statement’s retained interpretation.

4.2 Three verbs prevent three kinds of confusion

assume P introduces a hypothesis in a permitted logical scope. **derive** P asks for a proof from the current scope. **construct** x **satisfying** P asks for data and evidence under the current specification. They must not collapse into one ambiguous “let.”

In a theorem header, a binder such as “let $q : \mathbb{R}$ where $|q| < 1$ ” introduces both a universally quantified q and a hypothesis. Inside a proof, refining an already bound q with the same clause generates an obligation instead. A UI that makes these two uses visually identical without marking their role would be dangerous. SCHIST uses **assume** for the first and **derive** or a proof-bearing **with** clause for the second when the scope is not already explicit.

```
assume x : Real with x != 1      -- a local hypothesis
let y := (x^2 - 1) / (x - 1)
derive y = x + 1 by guarded_algebra

-- With arbitrary x, this does not silently add x != 1:
derive (x^2 - 1) / (x - 1) = x + 1 by guarded_algebra
-- Residual requirement: x != 1. The original goal stays open.
```

4.3 Reasons are either requirements or hints

A step written by **reindexing** $k = i+j$ requires a derivation through the registered reindexing interface. A step written **search**, **hint** **reindexing** permits another proof. This follows the second-round distinction between an advertised mathematical reason and an exploratory suggestion [1, 2].

Neither syntax requires a verbose protocol record in the ordinary document. The default display shows the claim, the meaningful reason, and any unresolved or meaning-changing conditions. Its expanded view shows the actual theorem, instantiated arguments, generated premises, and retained evidence.

4.4 Native Lean remains part of the language

An ordinary Lean **by** block is legal wherever a proof is required. Existing short declarations need no wrapper. A project can adopt just the property-aware binders, just a document view, or just the theorem packages. This is important for incremental use in repositories whose source is partly human-written and partly generated.

5 Four layers of mathematical typing

5.1 Layer I: carrier and computational structure

The first layer answers questions such as: what is the carrier; which ring operations are being used; which measurable structure is intended; what are the universe parameters; which module action occurs in a formula?

These choices can affect the mathematical meaning of an expression. They belong to ordinary Lean terms and instances and are fixed before proof search. Two ring structures on the same carrier are not interchangeable merely because the carrier has the same printed name. A proof of a proposition about one structure cannot be used for another without the corresponding transport theorem.

5.2 Layer II: proof-backed refinements of an existing object

For $a : A$ and $P : A \rightarrow \text{Prop}$, the notation

$$a : A \text{ where } P(a)$$

means that $a : A$ is retained and evidence of $P(a)$ is available. Multiple properties form an open row:

$$a : A \text{ where } \{P_1(a), \dots, P_m(a)\}.$$

The row is a local proof interface, not a new foundational intersection of unrelated carrier types. Adding $P_{m+1}(a)$ requires a proof. Forgetting an unused property does not change a .

A row may be viewed as a finite conjunction, but the implementation retains individual proof references and their dependency scopes. It should not eagerly construct an enormous conjunction or install every predicate as a global instance. For functions, regularity and algebraic laws remain ordinary properties: continuity on a set, monotonicity, linearity with specified operations, or a differential equation expressed through derivative certificates.

5.3 Layer III: relations and observations

Some information is not a unary property of a replacement value. Examples are

$$\text{EqOn}(U, f, g), \quad f =^{\mathcal{N}(x)} g, \quad \text{JetEq}_N(F, J), \quad \mu = \text{Law}(X),$$

and independence of X and Y under a fixed probability measure. These are propositions on several anchored objects. They belong in the same proof index, but they do not become unrestricted coercions between those objects.

An observation of a fixed formal series is still about that series. A finite jet is not another globally equal series; equality in law is not pointwise equality of random variables; two independent copies are not equal random variables. A transport rule must name the relation it consumes and the relation it produces. This extends the second-round representation discipline to the ordinary typing of mathematical steps [1, 2].

5.4 Layer IV: behavioral and operational contracts

A total pure function $f : A \rightarrow B$ may carry a specification

$$\forall x : A, \quad P(x) \longrightarrow Q(x, f(x)).$$

A global law may instead quantify several inputs or involve another function: monotonicity, naturality, injectivity, preservation of a distribution, or commutation with an action. A stateful program needs a specification of its state and observations, not just a proposition about its returned value.

These are all expressible with dependent types, but their proof obligations and useful automation differ. In particular, mathematical precision loss under differentiation is not a runtime side effect, and a library’s allowed axiom set is not an input precondition. SCHIST keeps these dimensions separate rather than encoding them in a single overloaded “effect” annotation.

Design decision. The four layers share evidence handling, not a universal coercion or a universal ordering of strength. Only the first layer determines the underlying operations; later layers describe proved information about them.

6 A conservative core calculus

6.1 Refinement formation and its two representations

Let $\mathcal{E}$ be an ordinary Lean environment and Γ an ordered local context. For $A : \text{Type}_u$ and $P : A \rightarrow \text{Prop}$, define the *first-class* refinement by the existing subtype construction:

$$\llbracket A \triangleright P \rrbracket = \{a : A \mid P(a)\}. \quad (2)$$

Lean already supports structures with data and proof fields. Its treatment of proof irrelevance and runtime-erased proof fields makes this a natural target; none of these mechanisms is new in this proposal [12].

Inside a proof, the preferred representation is *unbundled*:

$$a : A, \quad h : P(a).$$

Packing into a subtype occurs only where a first-class value is needed, and unpacking is explicit in the translation. The local name a remains the same Lean term. This avoids proliferating towers of subtype projections when a function gains additional properties during an argument.

For the basic core, row predicates may depend on earlier data but not on the choice of proof witnesses. General dependent records remain available through ordinary Lean syntax. In particular, data depending on an earlier chosen witness must remain in an ordered dependent telescope; they cannot be flattened into a commutative set of labels.

6.2 Judgments

Write

$$\mathcal{E}; \Gamma \vdash e \Downarrow a : A ; h : P(a)$$

when a source expression elaborates to $a : A$ and proof $h : P(a)$. A checking judgment receives A and P as demands; a synthesis judgment produces the base type and a bounded set of already justified properties. There is no claim that it synthesizes every true property of a .

The essential rules have ordinary Lean realizations:

$$\begin{array}{c} \frac{\Gamma \vdash a : A \quad \Gamma \vdash h : P(a)}{\Gamma \vdash (a, h) : A \triangleright P} \quad (\text{introduce}), \\[10pt] \frac{\Gamma \vdash v : A \triangleright P}{\Gamma \vdash v.\text{val} : A \quad \Gamma \vdash v.\text{property} : P(v.\text{val})} \quad (\text{project}), \\[10pt] \frac{\Gamma \vdash a : A \quad \Gamma \vdash p : P(a) \quad \Gamma \vdash q : Q(a)}{\Gamma \vdash a : A ; (p, q) : P(a) \wedge Q(a)} \quad (\text{merge evidence}). \end{array}$$

The last rule shares a . It does not combine two merely similar printed expressions or identify two independently chosen witnesses.

6.3 Subsumption is a theorem application

A sufficient rule for forgetting or changing a refinement is

$$\frac{\Gamma \vdash a : A ; p : P(a) \quad \Gamma \vdash k : \forall x : A, P(x) \rightarrow Q(x)}{\Gamma \vdash a : A ; k(a, p) : Q(a)}. \quad (3)$$

A point-specific implication $P(a) \rightarrow Q(a)$ also suffices. The uniform form is useful for reusable interfaces. Automatic property subsumption therefore means finding an ordinary proof of an implication, not declaring the two types definitionally equal.

For example, a proof of $0 < x$ can be used to supply $x \neq 0$ over the specified ordered field. It cannot supply invertibility in an arbitrary ring just because “positive,” “nonzero,” and “unit” are all useful adjectives in other settings. The ambient structure remains part of the proposition.

6.4 An extrinsic function contract

For an ordinary total function $f : A \rightarrow B$, write

$$\text{Contract}(f, P, Q) := \forall x : A, P(x) \rightarrow Q(x, f(x)). \quad (4)$$

Application uses both the function and its law:

$$\frac{f : A \rightarrow B \quad c : \text{Contract}(f, P, Q) \quad a : A \quad p : P(a)}{f(a) : B ; c(a, p) : Q(a, f(a))}.$$

Nothing about the value of f outside P is asserted by this contract. Nevertheless f itself is still total. This is deliberately distinct from an actual partial function whose domain is represented by a subtype.

Proposition 6.1 (Variance of extrinsic function contracts). *Suppose $c : \text{Contract}(f, P, Q)$, $r : \forall x, P'(x) \rightarrow P(x)$, and $s : \forall x y, P'(x) \rightarrow Q(x, y) \rightarrow Q'(x, y)$. Then f satisfies $\text{Contract}(f, P', Q')$.*

Proof. Given x and $p' : P'(x)$, apply c to x and $r(x, p')$, then apply s to $x, f(x), p'$ and the resulting postcondition proof. $\square$

Thus a function accepting more inputs can serve a caller offering fewer, and a stronger postcondition can serve a weaker demand. Confusing these directions is one reason not to assign contracts a single numerical strength rank.

6.5 Scope, assumption masks, and context embeddings

Every proof reference has a well-typed dependency context. A local fact proved in a branch with $h : x \neq 0$ is not available in a sibling branch that lacks h . Exporting it requires abstracting the assumption, producing $x \neq 0 \rightarrow P$, or discharging it. A user acknowledging a warning does not discharge it.

A cache lookup or an imported observation is accepted only after constructing a context embedding: a substitution mapping the retained parameters and hypotheses to terms of their required types in the current context. Matching names or hashing a printed goal is insufficient. Hashes remain useful fast indices, not mathematical evidence.

The implementation records the *allowed assumptions* for each pending obligation. A proof of a newly generated guard may use the earlier context, not the conclusion whose use generated that guard. Ordinary Lean’s dependency ordering provides the underlying logic; the new layer must preserve it while staging metavariables, asynchronous provider responses within a session, and editor transactions.

6.6 Properties cannot allocate new data invisibly

From $\exists x, P(x)$ one may introduce a witness inside a proof of a proposition by existential elimination. Producing a computational value for a public definition is a different request. It needs constructive data or an explicit noncomputable choice route with its dependencies recorded.

Similarly,

$$\forall h : G, \exists y, R(h, y) \quad \text{and} \quad \exists y, \forall h : G, R(h, y)$$

have different witness scopes. Even when proof irrelevance simplifies a particular proposition-indexed case, a generic request interface must not move witnesses across arbitrary dependent binders. SCHIST retains telescopes and makes the intended order explicit rather than inferring a stronger uniform construction.

7 Backward propagation of mathematical requirements

7.1 A method tells the elaborator what it needs

A reusable method consists primarily of an ordinary Lean theorem. Its metadata identifies semantic roles: input object, region, observation, requested postcondition, and proof obligations. Search priority and display choices are separate, versioned policy. This follows the stable-wrapper recommendation of the second round [1, 2].

The additional proposal is a *requirement transformer*. For a fixed total operation $f : A \rightarrow B$, a transformer Req_f maps a postcondition $Q : B \rightarrow \text{Prop}$ to a sufficient input condition. It comes with

$$\text{reqSound}_f : \forall Q \ a, \quad \text{Req}_f(Q)(a) \longrightarrow Q(f(a)). \quad (5)$$

The input condition is not necessarily weakest, and the transformer need not be available for every Q . In practice, a domain package recognizes a narrow family of demands and uses a theorem for each recognized form.

Proposition 7.1 (Composition of sufficient requirement transformers). *Let $f : A \rightarrow B$ and $g : B \rightarrow C$ have transformers satisfying (5). Then*

$$\text{Req}_{g \circ f}(R) = \text{Req}_f(\text{Req}_g(R))$$

is sufficient for $R(g(f(a)))$.

Proof. The first soundness theorem yields $\text{Req}_g(R)(f(a))$; the second yields $R(g(f(a)))$. No search, optimality, or decision-procedure assumption is used. $\square$

This is a simple logical principle, not a newly discovered foundation. Its language-design role is to make the author's next step determine which hidden premises deserve attention. Ordinary weakest-precondition reasoning is an important precedent, but mathematical methods also transform relations between objects, not just predicates on a single return value.

7.2 Relational requirements

For a representation change or mathematical theorem, the useful contract is more general:

$$\forall \vec{x}, \quad H_1(\vec{x}) \rightarrow \cdots \rightarrow H_k(\vec{x}) \rightarrow R(\vec{x}, \text{out}(\vec{x})).$$

Given a demand for R , the elaborator instantiates this theorem and generates $H_1, \dots, H_k$ in order. Some later H_i may depend on earlier constructed data. This is an AND/OR search graph with typed theorem applications as edges, not an untyped graph of expressions linked by a universal equality relation.

For the derivative example, the demand for a derivative transfer yields a requirement for germ equality. The demand for that germ can be met by equality on an open neighborhood and point membership. For a jet derivative at precision N , a registered rule requests input precision $N + 1$. For a law after a measurable pushforward, the probability-law demand produces the relevant measurability and original probability premises.

7.3 Why backward search is preferable to unrestricted property closure

Forward saturation would continually infer weaker continuity properties, nonzeroness of every product, all possible restricted domains, and every coherent representation of every object. Even if sound, it could be expensive and unpredictable. Many true properties would never be used.

The proposed default is demand-driven. Directly available facts and inexpensive registered consequences are indexed. A requested method then expands only the properties needed for that

method, under a fixed budget. Failed branches are rolled back. A proof of a useful new property can be cached after checking, but its existence does not authorize an unbounded closure pass.

The language may use a small, explicitly terminating fragment for routine entailment. Arbitrary semantic subtyping is not promised to terminate or to be complete. For a general proposition, the outcome may simply be an open obligation with a mathematical explanation.

7.4 The residual is a first-class diagnostic

The output of elaborating an incomplete step is not “type mismatch” alone. It is a residual specification, for example:

```
Cannot differentiate this equality at x.
Available: equality at x.
Required by the selected method: equality near x.
A sufficient route is equality on an open U containing x.
No additional hypothesis has been added to the theorem.
```

Such a residual explains a selected route, not a mathematical impossibility. Another method may prove the goal with different premises. The diagnostic must therefore say “required by this method,” not “necessary for the theorem,” unless necessity has separately been proved.

8 Behavior belongs in types, but not in one undifferentiated annotation

8.1 Pointwise postconditions are not whole-function laws

The bare type $\text{List}(A) \rightarrow \text{List}(A)$ permits the identity, reversal, duplication, constant empty output, and many other functions. A length-preservation contract excludes some of these, but does not specify a sorting function. Even over natural-number lists, the function returning as many zeroes as the input has elements is length-preserving and sorted; it is not generally a permutation of the input.

A suitable sorting specification therefore includes both order and multiset preservation:

$$\text{SortSpec}(f) := \forall xs, \quad \text{Sorted}(f(xs)) \wedge \text{Perm}(f(xs), xs). \quad (6)$$

A stable sort needs additional data about the comparison and the order of equal-key elements. A time bound is yet another contract. These are not consequences of a function’s carrier type or of finite tests.

A proposed declaration can state the entire behavior directly:

```
construct sort : List Nat -> List Nat satisfying
  for every xs:
    sorted (sort xs)
    permutation (sort xs) xs
  by synthesis using list_methods
```

The accepted product is a function f and a proof of (6). Synthesis is allowed to choose f , because the request delegates that choice. It is not allowed to weaken permutation to equal length. A candidate may be shown provisionally, but its specification remains open until proved.

8.2 Multi-input and higher-order behavior

Some laws cannot be decomposed into independent input and output refinements. Monotonicity states

$$\forall x y, \quad x \leq y \rightarrow f(x) \leq f(y).$$

Naturality relates an operation at two types and a connecting map. Independence is a relation between random variables under one joint probability measure. Equivariance relates the action on the input and the action on the output.

SCHIST permits a function-wide law field $L(f)$ beside pointwise pre/postconditions. An interface for an operation on functions may require $L(f)$ and return $M(T(f))$, using a theorem about the higher-order operation. It does not try to reduce every behavior to a refinement of one output at one input. This matters to Leant: a structural candidate can be well typed while violating a relational law that its structural search language does not model.

8.3 Actual partial functions and totalized operations

A partial mathematical function with domain $D \subseteq A$ can be represented by a function $\{x : A \mid D(x)\} \rightarrow B$. Alternatively, a package may use a relation describing definedness and value. This is distinct from a total $f : A \rightarrow B$ carrying a promise only when $D(x)$ holds.

Existing Lean expressions retain their existing total semantics. The optional partial-expression notation must mark its domain convention. In particular,

$$\frac{x^2 - 1}{x - 1} = 0$$

has different real solution sets under total division and under the partial reading that excludes $x = 1$. The corrected second-round analyses make this an explicit interface decision [1, 2]. SCHIST adopts total Lean semantics when importing Lean and requires an explicit partial-domain package for the alternative reading.

An equality of partial functions includes a domain statement. A simplification valid on a smaller domain is not a same-domain simplification. Extending a partial function across a removable singularity introduces an extension with a proved agreement relation; it does not retroactively alter the original object.

8.4 Operational behavior and total correctness

For a pure, total state transformer $c : S \rightarrow A \times S$, a contract may state

$$\forall s, \quad P(s) \rightarrow Q(s, (c(s)).1, (c(s)).2).$$

For a general operational semantics, partial correctness says that every terminating run from P satisfies Q . Total correctness additionally demands termination. Nondeterministic choice, exceptional exits, traces, resource use, and mutable state require their corresponding observations and quantifiers. They should not be inferred from a pure function specification.

Lean already has an explicit verification-condition framework around `mvcgen`, weakest-precondition reasoning, and monadic specifications. The proposed frontend should reuse it where applicable rather than claiming that effect-aware verification is absent from Lean [14, 15]. The new contribution would be a common author-facing contract interface and source-to-obligation map, with the monad-specific correctness theorem retained as a dependency.

A cost assertion also needs a declared cost model. A proof that a recursive algorithm takes at most k abstract steps is not a wall-clock timing guarantee. Similarly, proof-search fuel bounds the tool's effort, not the mathematical runtime of the function being synthesized.

9 Worked development: probability-law uniqueness without moment hypotheses

9.1 A reusable interface for characteristic functions

Let E have the hypotheses of ProveIt's inspected uniqueness theorem. For a probability measure μ , let $\chi_\mu : E \rightarrow \mathbb{C}$ be its characteristic function. The proof uses the following interface:

$$\chi_\mu(0) = 1, \quad \chi_\mu \text{ is continuous at } 0, \quad \|\chi_\mu(t)\| \leq 1.$$

Equality of the full characteristic functions implies equality of the measures in the specified setting. The source supplies this through its library imports and theorem applications [4].

The characteristic function remains anchored to μ . A generic function satisfying the three displayed properties is not automatically the characteristic function of a probability measure. In particular, SCHIST does not infer existence of a law from this short profile. The extensionality method applies to characteristic functions of the original measures, not to arbitrary profile values.

The package can expose a `characteristic_profile(mu)` observation together with the exact relation to μ , and derive the particular properties demanded by a proof. If only continuity at zero is needed, it supplies that consequence; it need not insert a stronger regularity assumption into the theorem.

9.2 The mathematical argument, in full

Assume $|q| < 1$, and suppose two probability measures satisfy

$$\chi_\mu(t) = a(t)\chi_\mu(qt), \tag{7}$$

$$\chi_\nu(t) = a(t)\chi_\nu(qt), \tag{8}$$

where $\|a(t)\| \leq 1$. Define

$$d(t) = \|\chi_\mu(t) - \chi_\nu(t)\|.$$

Subtracting the two recurrences and using multiplicativity of the complex norm shows

$$d(t) = \|a(t)\|d(qt) \leq d(qt).$$

Inductively, for every $n \in \mathbb{N}$,

$$0 \leq d(t) \leq d(q^n t). \tag{9}$$

Since $q^n t \rightarrow 0$, continuity at zero and the common normalization imply

$$d(q^n t) \longrightarrow \|1 - 1\| = 0.$$

Taking the limit in (9) gives $d(t) = 0$. This holds for every t , hence $\chi_\mu = \chi_\nu$ and then $\mu = \nu$.

For the affine law (1), the common multiplier is $a(t) = \chi_{\rho \text{ map } d}(t)$. Measurability of d establishes that this pushforward is a probability measure, which supplies the bound on a . The characteristic-function formula for convolution and scaling supplies (7)–(8). This is the same proof route as the inspected source, presented through a reusable interface [4].

There is no cancellation of $a(t)$; it may vanish. There is no moment, density, compact-support, or absolute-continuity premise. There is also no existence conclusion: the result says that there is at most one probability fixed point.

9.3 A general orbital-comparison theorem

The argument suggests a small reusable theorem that separates its real content from the characteristic-function representation. This formulation is a generalization made here for interface design; no claim of historical novelty is made.

Theorem 9.1 (Orbital comparison). *Let X be a topological space, $T : X \rightarrow X$, and $p \in X$. Suppose for every $x \in X$ the iterates $T^n(x)$ converge to p . Let $f, g : X \rightarrow \mathbb{C}$ be continuous at p , with $f(p) = g(p)$. Suppose $a : X \rightarrow \mathbb{C}$ satisfies*

$$f(x) = a(x)f(Tx), \quad g(x) = a(x)g(Tx), \quad |a(x)| \leq 1 \quad (x \in X).$$

Then $f = g$.

Proof. Fix x . The two recurrences give $|f(x) - g(x)| \leq |f(Tx) - g(Tx)|$. Induction gives the same inequality with $T^n x$ on the right. By continuity at p , the right-hand side tends to $|f(p) - g(p)| = 0$. A nonnegative real number bounded above by every member of a sequence converging to zero is zero. Thus $f(x) = g(x)$, for arbitrary x . $\square$

Neither continuity of T nor global continuity of f, g is required once the orbit-convergence hypothesis is supplied. The theorem is a good example of a contract that preserves exactly the generality of the argument rather than requesting a familiar but unnecessarily strong class of functions.

There is a useful optional refinement. Define

$$A_n(x) = \prod_{j=0}^{n-1} a(T^j x), \quad A_0(x) = 1.$$

If for each x there is a finite $M_x \geq 0$ with $|A_n(x)| \leq M_x$ for all n , the same conclusion follows without the pointwise bound $|a(x)| \leq 1$. Indeed, iteration gives

$$|f(x) - g(x)| = |A_n(x)| |f(T^n x) - g(T^n x)| \leq M_x |f(T^n x) - g(T^n x)| \rightarrow 0.$$

This is a different sufficient method contract. The original pointwise bound implies it with $M_x = 1$; a method registry may offer either route. It should not assert that the first route's hypotheses are necessary for uniqueness.

9.4 The proposed proof and its elaboration

```
show mu = nu by characteristic_functions
  let f := characteristic(mu)
  let g := characteristic(nu)
  let a := characteristic(map digit rho)

  derive f(t) = a(t) * f(q*t) for all t from the fixed-point law
  derive g(t) = a(t) * g(q*t) for all t from the fixed-point law

  show f = g by orbital_comparison at 0 along (t => q*t)
```

The final line requests the premises of Theorem 9.1. Its requirement graph asks for convergence of $q^n t$, continuity of f, g at zero, equal values there, the two recurrences, and the norm bound on a . Each demand has a specific source: the scalar contraction hypothesis, the probability-law profiles, or the already stated fixed-point equations.

The proof is short because these properties are reusable parts of the objects' interfaces. It is not short because `orbital_comparison` secretly proves the entire probability theorem by arbitrary

automation. Its theorem and its premises are inspectable, and ordinary Lean can use the same wrapper.

9.5 Negative neighbors that test the type layer

The strict contraction hypothesis matters. With $q = 1$ and $a = 1$, both recurrences become tautologies for every characteristic function; distinct probability measures need not be equal. With a constant zero digit, the affine operator itself is the identity when $q = 1$.

Normalization also matters to the generic profile argument: two different constant complex functions are continuous and satisfy the recurrence with $a = 1$, but are not equal. For characteristic functions of probability measures, the common value at zero is derived, not guessed.

A route using a finite-moment Wasserstein space may be a valid alternative under additional hypotheses, but cannot discharge this theorem by silently adding those hypotheses. A source-level contract linter should flag such a change even if the stronger statement has a valid proof. Finally, replacing “at most one” by “there exists exactly one” requires an existence theorem; no uniqueness method can manufacture it.

10 Locality, induction motives, and mathematical proof plans

10.1 A region is part of the contract

For functions on $\mathbb{R}$, define

$$\text{EqOn}(U, f, g) := \forall x \in U, f(x) = g(x).$$

There is a proved bridge from $\text{EqOn}(U, f, g)$, openness of U , and $x \in U$ to equality in a neighborhood of x . There is no such bridge from the single equality $f(x) = g(x)$. The functions $f(t) = t$ and $g(t) = 0$ at $x = 0$ are the standard counterexample.

The type of a differentiation step should demand the correct local relation. A user may write “differentiate the identity on U ,” but the expanded contract specifies the derivative certificates and the locality needed at each point. At a boundary, a within-set derivative may be the right relation; it must not be silently changed into an ambient derivative.

10.2 An induction hypothesis has a scope, not just a formula

In the autonomous-derivative example, the induction statement is

$$I(n) := \forall x \in U, W^{(n)}(x) = G_n(W(x)).$$

Fixing one x too early would produce a weaker induction hypothesis that cannot supply neighborhood equality. A proof-plan interface can request the correct motive and explain why the point variable must remain generalized.

A possible plan is:

```

prove for every n:
  on U, iterated_derivative n W = G(n) composed_with W
by induction n keeping the point variable general
  base: use the initial equation for G(0)
  step:
    differentiate the induction identity on U
    use the chain rule and the recurrence for G

```

The system does not need to infer arbitrary induction motives. This plan names a specific theorem schema. Leant may construct adapters among its premises, but the chosen generalized statement is fixed and displayed before those adapters are accepted.

10.3 Range-local requirements

The inspected source requires derivatives of G_n only at $W(x)$ for $x \in U$ [3]. A refinement of the family may therefore be indexed by `along W(U)` rather than `everywhere`. A wrapper for the chain rule can consume exactly that contract.

This prevents two opposite mistakes. Demanding global regularity needlessly strengthens the theorem. Demanding only arbitrary pointwise equalities weakens the premises beyond what differentiation permits. The type layer’s role is to retain the relevant region and relation while letting their routine proofs be implicit.

11 The Frey-package case: mathematical structure without instance ambiguity

11.1 Relational refinement of a tuple

A mathematical declaration for the existing Frey package could read:

```
assume (a, b, c, p) : Int * Int * Int * Nat satisfying
  a != 0, b != 0, c != 0
  prime p and 5 <= p
  a^p + b^p = c^p
  gcd(a,b) = 1
  a = 3 modulo 4
  b = 0 modulo 2
```

This is not proposed as a replacement for the existing structure. A package adapter can unpack `P : FreyPackage` into these exact fields, and repack the fields into the structure when an existing theorem expects it. The tuple constraint is a dependent record of data and relations, not four independent unary refinements.

Deriving that p is positive or odd, or that $abc \neq 0$, is then a demand-driven application of the package’s ordinary lemmas. Constructing such a package from an arbitrary counterexample remains a substantial mathematical normalization argument; the type annotation does not prove it.

11.2 The terminal contradiction should stay terminal

The inspected terminal proof has the mathematical shape

```
let irreducible := Mazur_Frey P
let modular := frey_isModular P
obtain f with f != 0 from level_lowering_to_two P modular irreducible
contradict f != 0 using S2_Gamma0_2_eq_zero f
```

This is a proposed readable projection of the existing small proof body, not a measured compression result [7]. Native Lean may be equally good or better here. The useful extension is that the typed display expands the exact meanings of `irreducible`, `modular`, and the cusp-form space on demand.

The repository’s proof-path guide explicitly distinguishes its specialized statements from more general classical theorems. For example, its level-lowering route concerns the Frey representation and trace congruences in the specified setting; its irreducibility terminology does not by itself mean absolute irreducibility. Such distinctions must survive any compact mathematical name [6].

An interface should therefore be keyed to a theorem’s exact declaration and contract, not to the informal label “Ribet” or “Mazur.” The intended mathematical explanation can be retained even when a proof term has an unused formal premise, but the display should distinguish an advertised dependency from a dependency actually used by that term. The guide notes an example where a level-lowering premise is formally unused because modularity is rederived inside the proof [6].

11.3 A lexical structure environment

The solution file’s generated instance-control preamble motivates a separate feature: a *lexical structure environment*. It fixes the relevant carrier, operations, scalar actions, and instance dictionaries for a block, using ordinary explicit arguments and scoped Lean mechanisms in the translation.

This is not a new global instance-selection oracle. The chosen dictionaries are part of the elaborated meaning. Property inference may discover a proof that an existing structure satisfies a law; it may not silently choose a different structure to make a theorem applicable. Competing choices require an explicit route or a theorem showing that the observations used by the proof are preserved.

Some instance preambles may still be necessary for generated code, and it has not been established that this feature would remove them. The implementation experiment must distinguish a cleaner display from reduced elaboration cost and from genuinely simpler source management.

11.4 Integral models are not rational models by typography

Suppose an integer coefficient is written $m/4$ and the corresponding rational coefficient is $(m : \mathbb{Q})/4$. Their agreement after casting follows under an appropriate divisibility hypothesis; it is false in general. For example, integer division of 1 by 4 is zero, while rational division gives $1/4$.

A model-transport method should request the divisibility evidence from the normalization contracts and preserve the exact map between the two curve objects. This is a promising interaction between refinements, arithmetic synthesis, and representation theorems. It is not a reason to add rational normalization to definitional equality.

12 Certified algebra as a client of the type layer

12.1 Typed requests and typed results

The inherited computational interface is

$$\text{Request}(x, O, \text{Spec}), \quad \text{Answer} = \{(y, p) \mid y : O(x), p : \text{Spec}(x, y)\}.$$

A conditional result carries a proof $G \rightarrow \text{Spec}(x, y)$; it is not an accepted unconditional answer until G is established at the caller’s scope. A result whose data depend on a guard or an earlier witness retains that dependence in its type [1, 2].

The property layer helps in two places. Before computation, it supplies the algebraic assumptions and input-domain conditions. After computation, it supplies the consumer with the proved properties of the returned data without forgetting which original object they describe.

12.2 A polynomial identity is not every algebraic conclusion

A certificate

$$p = \sum_{i=1}^m q_i f_i$$

proves $p = 0$ from $f_i = 0$ after interpretation in the specified commutative ring. It does not, by itself, prove irreducibility, completeness of a solution list, or a negative ideal-membership claim. A certificate for $p^k = 0$ needs an appropriate hypothesis to infer $p = 0$.

The contract must also distinguish a unit from a regular element and from a merely nonzero element. In $\mathbb{Z}$, 2 can be cancelled from an equality but has no multiplicative inverse. In $\mathbb{Z}/4\mathbb{Z}$, 2 is nonzero but cannot generally be cancelled. These are separate property demands, even if a higher-level UI places them under one algebraic menu.

Existing proof-producing tactics are controls, not obstacles. The syntheses report successful uses of `linear_combination` and the bounded `grobner` tactic in their pinned experiments. A new normalizer must justify its additional value, and an unsuccessful bounded tactic call is not a refutation [1, 2].

12.3 Jet precision is an index, not an exactness badge

For formal series over a commutative ring, put

$$\text{JetEq}_N(F, J) : \Longleftrightarrow \forall k < N, \quad [t^k]F = [t^k]J.$$

A derivative observation at precision N can be obtained from input precision $N + 1$. The demand transformer therefore propagates $N + 1$ backward. A cached higher-precision observation can serve a lower-precision demand through a proved restriction theorem. It cannot serve an unbounded identity request.

For a residual certificate, let $H(Y) \in R[[t]][Y]$, $H(F) = 0$, and $F(0) = d$. If the reduction $\overline{H}'(d)$ is a unit, $J(0) = d$, and $H(J)$ vanishes modulo t^N , then F and J agree modulo t^N , for $N \geq 1$. The proof factors $H(J) - H(F) = (J - F)U$ and uses the unit constant coefficient of U . This theorem is inherited from the round-two analysis, not a new result here [1, 2].

Its contract exposes the existing root, the common branch, the unit, and the precision. A polynomial producer may discover J , but the observation remains indexed by the original F . Removing the unit condition fails over $\mathbb{Z}/4\mathbb{Z}$ with $H(Y) = 2Y$, $F = 0$, and $J = 2t$. Removing the common constant term fails for the two roots of $Y^2 - 1$.

12.4 Complete solving requires both directions

A complete solution predicate S for P requires

$$\forall x, \quad P(x) \longleftrightarrow S(x).$$

This consists of both soundness and coverage. A single verified witness and a coverage assertion do not establish that every advertised solution is valid.

For $ax = b$ over a field, the complete predicate must handle $a \neq 0$, $a = b = 0$, and $a = 0 \neq b$. A function contract returning b/a only under $a \neq 0$ cannot be specialized at $a = 0$ while pretending that its guard disappeared. The property layer may branch and establish complete coverage, or leave a conditional answer; it may not change the requested guarantee.

12.5 The execution boundary remains explicit

A theorem about a verified algorithm is not evidence that arbitrary bytes returned by an external executable are its result. A sound checker requires both a theorem connecting its accepted

certificates to the semantic claim and a justified evaluation of the particular checker invocation. Source reification must also connect the finite input to the original Lean expression [1, 13].

SCHIST initially uses ordinary proof-producing reconstruction or kernel reduction for the small computational core. Accelerated mechanisms are optional trust profiles with actual transitive axiom inventories. Their adoption requires measurement on the real workload and a policy decision, not an assumption that native execution is always faster. No solver participates in foundational conversion.

13 Leant as a supplier of evidence candidates

13.1 The current baseline is richer than bare type inhabitation

The current Leant documentation describes exact semantic-origin records, provider bindings, typed candidate associations, bounded verification, and configurable candidate ordering. The inspected verification module deliberately makes its **Verified** wrapper mean acceptance by the supplied callback, not behavioral evidence or a kernel proof [9, 8].

The current Length layer is also more than the earlier idea of merely attaching Z3. It offers opt-in behavior-sensitive ranking and explicitly authorized filtering, with independently replayed counterexamples and bounded progression through candidate batches. Raw solver statuses do not acquire positive proof authority. Its documentation distinguishes this implemented slice from full counterexample-guided synthesis, additional behavioral domains, and Lean-checked behavioral artifacts, which remain proposed work [10].

The earlier behavioral-synthesis proposal already advocates typed sketches, narrow semantic domains, approximation directions, and final Lean proofs [11]. Those are not claimed as inventions of SCHIST. The new integration point is the mathematical property-demand graph: it supplies specific, scoped contracts to that machinery and consumes its results as ordinary typing evidence.

13.2 Three different requests

Proof construction. Given a fixed proposition P in an exact Lean context, ask for $p : P$. A structural engine can compose the available theorem providers, instantiate arguments, and construct conjunctions, implications, or witness packaging. The exact term must be checked against the original P , not merely against an abstraction of its shape.

Data construction with behavior. Given A and $\text{Spec} : A \rightarrow \text{Prop}$, ask for $a : A$ together with $\text{Spec}(a)$. The request delegates the data choice but not the specification. For a synthesized function, this is a global law such as (6), not just type inhabitation followed by a reassuring ranking score.

Proof-edit suggestion. Given an editor proof state, ask for a concrete edit. A successful probe and a displayed edit are separate artifacts. The exact edit must be replayed from its origin before it is called verified, with remaining goals, unresolved metavariables, completion, and trust-policy status reported separately. This is a requirement carried forward from the syntheses; it is not a claim that this article audited every current path in `Main.hs` [1, 2].

13.3 A concrete adapter protocol

A request contains the exact environment identity, ordered local context, fully elaborated target or dependent output specification, frozen semantic choices, allowed providers, budget, and trust policy. It distinguishes proof holes from explicitly delegated data holes. Lean-side handles identify original expressions; a printed name is not an identifier for mathematical authority.

The provider returns a candidate term, a candidate certificate, a proposed edit, or a structured non-success outcome. The Lean-owned boundary then:

1. reconstructs the candidate at the retained original context and target;
2. checks all required proof obligations and the final declaration;
3. audits actual dependencies under the selected policy;
4. retains sufficient evidence for search-free replay; and
5. commits an editor change only if its exact origin is still valid.

The transport can initially use the existing Haskell service. Nothing in the logical contract requires it to remain a separate process or forbids a later Lean-hosted implementation. The mathematical authority is the checked term, not the language in which the search engine is implemented.

13.4 Behavior-guided search without behavior laundering

Consider a candidate f for (6). A counterexample to permutation at a concrete list is useful for rejecting that candidate. A checked Lean proof of the concrete failure can refute the universal specification. An independently replayed counterexample in a modeled Length domain has the narrower authority of that domain unless the source-to-model bridge needed for the stronger claim is supplied.

Likewise, an SMT answer `unsat` for an abstract query is not automatically a Lean theorem. One safe route is to reconstruct a proof in the modeled fragment and compose it with a proved adequacy theorem at the original target. Another is to use the solver only to guide candidate selection and discharge the final universal law in Lean. A finite-box exhaustive check establishes only the finite-box claim.

For recursive functions, examples can guide an invariant or induction plan, but do not replace induction. A supplied invariant must be initialized, preserved by every relevant branch, and shown to imply the requested postcondition. Termination and computational executability have their own requirements.

13.5 Where structural synthesis should help

The useful population of requests is concrete, not “all theorem proving.” Examples include composing measurability and pushforward laws to obtain a probability-law property; lifting an equality on a region to the local relation needed by a derivative theorem; supplying the ordered premises of a representation bridge; assembling soundness and coverage into a complete solution predicate; and packaging a witness with its already proved laws.

Single-lemma lookup is an important baseline. If the same task is solved by a single indexed theorem application, the structural engine should not receive credit for a novel composition capability. Conversely, bounded composition of several small existing interfaces is exactly where Leant can reduce the manual adapter code between mathematical steps.

13.6 Negative results must name their logical strength

The protocol distinguishes unsupported input, resource exhaustion, failure within a bounded search, non-derivability in a specified calculus, a modeled counterexample, and a checked refutation of the original target.

Non-derivability is not negation. A complete search procedure for an intuitionistic fragment can fail to derive excluded middle without deriving its negation. To promote a refutation of an abstraction A to a refutation of P , one needs both a proof of $\neg A$ and the appropriate implication $P \rightarrow A$. Completeness metadata or a safety flag is not a replacement for that bridge [1, 2].

14 Which type-system extension should actually be implemented?

14.1 Option A: more named bundled structures only

This is already possible and often excellent. The Frey package is a useful example. Bundled objects provide stable interfaces, enforce invariants at construction, and support conventional library APIs. They should remain the preferred representation when the bundle itself is mathematically meaningful or widely reused.

Their limitation is not expressiveness. Repeated ad hoc conjunctions of properties can require boilerplate structures or nested subtypes; several bundles may wrap the same underlying function with different projections; and a fact proved in the middle of an argument may not fit an existing nominal bundle. Open local refinements address these use cases without abolishing bundled mathematics.

14.2 Option B: an elaboration-level refinement and contract system

This is the recommended initial implementation. Add property binders, relational clauses, proof-backed subsumption, requirement-driven method elaboration, and a visible residual-obligation interface. Translate them into ordinary Lean contexts, subtypes, structures, and theorem applications.

The extension has real typing rules at the authoring level, but no new kernel axioms or conversion rules. It can preserve compatibility with Mathlib and existing theorem statements. Most of the implementation can live in a Lean package: syntax elaborators, environment extensions for method metadata, scoped proof indices, and source-to-evidence maps.

14.3 Option C: primitive refinement types in the kernel

One could add primitive syntax and judgmental rules for a refined type, proof-erasing projections, or intersections of predicates over one carrier. Such an extension would need a precise metatheory covering substitution, conversion, universes, elimination, positivity interactions, and proof erasure.

If the proposed primitive is faithfully encoded by existing subtypes and proof fields, its additional value would be representational or ergonomic, not new mathematical expressiveness. That value should be measured before changing the trusted foundation. If it is *not* faithfully encoded, the exact new principle must be stated and justified rather than described vaguely as a more mathematical type system.

14.4 Option D: semantic equality inside conversion

Allowing arbitrary theorem proving, CAS normalization, or semantic subtyping to decide conversion would couple foundational checking to expensive search and to choices about domains, branches, and algebraic structures. Equality reflection is a much larger foundational change than property-aware elaboration. It is not needed for the examples in this article.

SCHIST therefore keeps the distinction between definitional conversion and propositional transport. Certified normalization may insert a proof of equality; it does not make all mathematically equal expressions kernel-convertible. Failure to find a subsumption proof leaves an obligation instead of changing the foundation's equality relation.

14.5 Relation to refinement-type research

Refinement systems and proof-irrelevant interpretations have a substantial history. Lovas and Pfenning's refinement system for logical frameworks is a particularly relevant precedent for refining existing terms while controlling the interaction with proof irrelevance [16]. Its metatheory and

decidability results are for its specified calculus, not a license to claim that arbitrary mathematical predicates over Lean admit decidable inference.

Mizar-style mathematical typing is an even closer precedent. Kaliszyk and Pāk give a mechanized account with type intersection, inhabitation reasoning, type inference, and definition-specific proof obligations [17]. Isabelle’s locales also provide scoped parameters and assumptions for reusable abstract reasoning [18]. Neither mathematical property typing nor persistent local proof contexts should therefore be advertised as new.

The distinguishing target is narrower: integrate these ideas into Lean while retaining the exact existing term, keeping local and finite observations indexed by their actual relation, and using the resulting demands as contracts for Leant and certified computation. Whether that combination improves on ordinary Lean contexts and theorem packages is precisely the proposed experiment.

The novelty sought here is consequently an engineering and language-design combination: open local mathematical interfaces, exact-object observations, region-aware requirements, and integration with current Lean and Leant. It is not the invention of refinement types, Hoare logic, or proof-carrying computation.

15 Soundness, identity preservation, and their limits

15.1 A relative preservation statement

The core specified in Section 6 consists of ordinary typed terms, refinement introduction and projection, row conjunction, proved subsumption, contract application, and ordered composition through registered theorems. Its translation is defined by those explicit constructors and applications.

Theorem 15.1 (Relative preservation for the specified core). *Assume an ordinary well-formed Lean environment $\mathcal{E}$, a well-formed translation of Γ , and ordinary Lean proofs for every registered method and bridge used by a finite core derivation. Then translating a core derivation*

$$\mathcal{E}; \Gamma \vdash e \Downarrow a : A ; p : P(a)$$

produces a Lean term of A and a Lean proof of $P(a)$ in the translated context. At a refinement-only or subsumption step, the underlying value remains a . No new assumption is introduced except at an explicit logical assumption rule.

Proof. Proceed by induction on the finite derivation. Base terms are typed in the assumed Lean context. Refinement introduction translates to a subtype constructor, and projection to its existing projections. Row merge translates to conjunction introduction over the same term. Subsumption is application of its supplied implication proof to the retained value and property proof. Contract application is ordinary universal instantiation followed by implication elimination. Ordered composition first introduces the actual intermediate value, then applies the next theorem to that value and its proof, preserving all dependencies. Context abstraction translates to the corresponding Lean binder; context reuse requires the typed substitution specified by its embedding. These cases use only existing Lean typing rules. Inspection of the refinement and subsumption translations shows that their value component is unchanged. No other case introduces a hypothesis. $\square$

This is a written metatheoretic argument for the stated small calculus. It is not a mechanically checked theorem about a future parser, elaborator, renderer, provider scheduler, or editor. Those components can still contain bugs. The exported declaration must therefore be checked against the frozen original target, and a high-assurance publication workflow should independently validate it and inspect its axioms as described by Lean’s validation guidance [13].

15.2 Proof irrelevance does not imply representation coherence

If two proofs establish the same property of the same fixed object, choosing one rather than the other need not change the object's value. That is the useful coherence property for extrinsic refinements. It does not imply that all rings on one carrier, all selected roots of one polynomial, or all representatives of a quotient are interchangeable.

Nor does it imply that two explanations of a theorem are editorially identical. A proof by a named geometric method and a proof by exhaustive calculation may have the same proposition but different explanatory roles. The source contract for an advertised method remains separate from proof irrelevance in the logic.

15.3 A closed theorem cannot acquire a hidden guard

Let a theorem request fix a fully elaborated type T . Suppose a selected method generates G and can prove $G \rightarrow T$. Without a proof of G , the result has type $G \rightarrow T$, not T . The interface may retain this conditional progress but cannot mark the original request complete. Replacing the theorem header by $G \rightarrow T$ is an explicit statement edit.

This simple observation is the central safety property of requirement-driven typing. It also protects against silent upgrades of an additive group to a vector space, a ring to a field, a formal identity to an analytic one, or a uniqueness result to an existence-and-uniqueness result.

15.4 What remains undecided by these theorems

The preservation argument does not establish completeness of elaboration, existence of a principal refinement, automatic discovery of weakest conditions, optimal proof search, minimal visible explanations, or agreement between a reader's informal intention and an expanded formal statement. It establishes only the logical meaning of the specified translation rules.

In particular, a semantically faithful display is a separate engineering obligation. A proof checker can accept a perfectly valid theorem that a renderer misdescribes. SCHIST generates its formal statement view from the retained typed objects and shows meaning-bearing conditions by default, but reader comprehension must still be evaluated.

16 A concrete implementation slice and reference core

16.1 The smallest useful vertical slice

The initial slice should implement ordinary mathematical binders with proof-backed property rows, one method with ordered requirements, and export of the resulting ordinary Lean declaration. It should exercise both a positive case and a close negative neighbor, and it should work with the discovery provider disabled during replay.

A practical sequence is: an affine-arithmetic checker and source bridge; conditional arithmetic using a nonzero premise; one region-aware derivative adapter; and the orbital-comparison package applied to the probability case. The first tests the computation boundary, the second the guard policy, the third locality and induction interfaces, and the fourth a nonalgebraic mathematical workflow. Each package is independently usable in Lean.

This does not discard the round-two residual-jet recommendation. The jet package is a natural additional client once the first checker and property-demand protocol are established. It should not delay a small end-to-end demonstration of the new typing feature.

16.2 An explicit small checker rather than an unspecified verified CAS

The accompanying `SchistCore.lean` defines refinements and their value-preserving weakening, extrinsic function contracts and consequence, anchored observations with ordered composition, and sufficient requirement transformers. It also gives a deliberately tiny affine-expression language over $\mathbb{N}$:

$$e ::= n \mid x \mid e_1 + e_2 \mid n \cdot e.$$

Its normalizer returns (a, b) , intended to mean $ax + b$. The equations are

$$\begin{aligned} \text{nf}(n) &= (0, n), & \text{nf}(x) &= (1, 0), \\ \text{nf}(e_1 + e_2) &= (a_1 + a_2, b_1 + b_2), & \text{nf}(n \cdot e) &= (na, nb). \end{aligned}$$

Proposition 16.1 (Affine normalizer denotation). *For every expression e and $x \in \mathbb{N}$, if $\text{nf}(e) = (a, b)$ then $\text{eval}(e, x) = ax + b$. Consequently, equality of normal forms implies equality of the interpreted expressions for every $x \in \mathbb{N}$.*

Proof. Structural induction on e . Literals and the variable are immediate. Addition uses $(a_1x + b_1) + (a_2x + b_2) = (a_1 + a_2)x + (b_1 + b_2)$. Scaling uses $n(ax + b) = (na)x + nb$. The consequence follows by applying the denotation identity to both expressions with their shared pair. $\square$

The Lean source writes the corresponding `eval_nf` and `check_sound` proofs, with `check` deciding equality of the two natural-number pairs. It also connects a closed certificate to the original statement

$$\forall x : \mathbb{N}, \quad 3(x + 2) = 3x + 6.$$

Here source reification is definitional because the expression is constructed explicitly; a general parser or reifier is not implemented. Changing the final constant to 7 gives a rejected certificate.

Evidence status. The Lean source contains no authored axioms, `sorry` placeholders, or `native_decide` calls, but it was not compiled in this preparation environment. Its elaboration and transitive axiom inventory remain to be checked. The file contains the intended `##print axioms` commands. The written mathematical proof above is independent of successful compilation.

This small example is not a competitive algebra engine. It is useful because its denotation, certificate predicate, soundness argument, and original-target connection are all explicit. It prevents the architectural discussion from hiding the very bridge the second-round syntheses found missing.

16.3 The elaboration algorithm

An initial implementation can use the following bounded workflow:

1. Elaborate and freeze the statement, structures, and delegated choices.
2. Parse the chosen mathematical step and resolve its method wrapper.
3. Instantiate its conclusion against the fixed demand.
4. Generate its ordered premises in the exact local telescope.
5. Try existing evidence, registered direct rules, then bounded composition.
6. Check every returned proof and keep unresolved premises as a residual.
7. Reconstruct and check the complete original-target declaration.
8. Audit dependencies, retain evidence, and commit only at the same origin.

The search state is transactional. Branch-local metavariable assignments do not leak into other routes. Shared data choices must be checked for compatibility before commit. Candidate generation is bounded independently of certificate size and checking cost, which also require resource limits.

A property cache stores proofs, exact anchors, dependency scopes, and relevant interpretation identities. It may propose a cached proof at a new request, but a typed context embedding and a guarantee-entailment theorem decide whether reuse is valid. A larger-domain identity can restrict to a smaller domain; a higher-precision jet can restrict to a lower precision. Additional hypotheses or additional axioms are not automatically acceptable at the new request.

16.4 What state is actually new

Ordinary parameters, hypotheses, and local instances remain Lean’s context. The extra state is limited to semantic role metadata, links from source steps to evidence, bounded indices for demanded properties, retained method choices, and publication/replay receipts. The design should not duplicate every `have` declaration into a competing logical store.

This allocation responds directly to the second-round criticism of overbuilt workspaces. Any extra index must justify an operation such as better diagnostics, proof reuse, or stable editing, and its cost belongs in the benchmark [1, 2].

17 Publication, maintenance, and semantic visibility

17.1 The published mathematical statement is not a pretty paraphrase

The default view must retain the quantified objects, relevant structures, domains, branches, precision, completeness, and unresolved conditions. Routine proofs can be collapsed. An interval answer must look like an interval answer; a conditional identity must not look unconditional; a finite jet must display its order.

For a function with a behavioral contract, the view includes the accepted function definition or a stable reference to it and the exact law established. A label such as “verified sort” is insufficient without the sorting specification. For a probability result, “same law” must not render as “same random variable.”

Every formal assertion in a published proof document must be checked, including an unused intermediate assertion. A separate valid proof of the final theorem does not justify publishing an incorrect explanatory step [1, 2].

17.2 Replay and migration are different operations

At a pinned environment, replay uses retained terms or certificates and their checked interpretation bridges, without rediscovering the proof. This can still involve substantial computation. An editable tactic script may accompany the authoritative evidence, but search instructions alone are not a durable proof artifact.

A toolchain or library upgrade requires a new check and a new receipt. The migration examines the resulting theorem statement, the definitions and structures it depends on, the relation of retained observations to their anchors, and the destination trust policy. Matching a printed theorem name or renaming an axiom in an allowlist is not revalidation. The syntheses explicitly identify native-evaluation axiom changes as a migration test [1, 2].

17.3 Merging two authors’ work

Two branches may add compatible proof-backed properties of the same fixed object; their merge can retain both after checking dependencies. If they change the selected object, instance dictionary, branch convention, or specification, the conflict is semantic, not just textual. Rechecking both branches’ old proof terms does not resolve the question of which statement the merged document intends.

A useful initial merge experiment therefore changes one property’s proof on one branch and its region on another, then checks which retained facts remain valid and what the reader sees. There is no claim that arbitrary semantic merge conflicts admit automatic resolution.

18 Evaluation that can reject the proposal

18.1 Separate five treatments

A fair study uses the same underlying mathematical assets:

Arm	Available facilities
L0	Existing Lean and library facilities.
L1	Lean plus the new theorem and definition packages, including orbital comparison.
L2	L1 plus the same Leant and certified-computation services.
L3	L2 plus property-aware binders, requirement propagation, and residual diagnostics.
L4	L3 plus the narrative mathematical step syntax and document view.

The important new comparison is L3 against L2: does the property-typing layer save effort once ordinary Lean receives exactly the same theorem wrappers and search services? L4 against L3 then tests narrative syntax instead of attributing all library gains to a language. A document-only variant can be a separate presentation condition.

18.2 Tasks and costs

Development tasks include the inspected derivative theorem, probability-law uniqueness, Frey-package property extraction and model transport, guarded algebra, and a residual-jet observation. They are not held-out tasks. A separate evaluator should choose related but unseen tasks, excluding aliases and close downstream restatements of the development examples.

Measure time to a correctly stated checked theorem, author interventions, failed attempts, source edits, checking cost, certificate size, maintenance work, and reader understanding. Record library construction, theorem-wrapper maintenance, checker proofs, and method metadata as infrastructure costs rather than treating them as free.

For repeated tasks, one transparent amortization model is

$$C_k = I + \sum_{j=1}^k (A_j + M_j),$$

where I is shared infrastructure effort, A_j authoring effort, and M_j maintenance effort under prescribed edits. This is bookkeeping, not an empirical savings claim. Report raw components as well as totals, because infrastructure effort may transfer to multiple projects.

18.3 Paired adversarial tests

Each negative case needs a nearby valid case. Rejecting everything is not a successful type system.

Pair	Positive case	Negative mutation and required outcome
Local differentiation	Equality on an open neighborhood and derivative evidence.	Only equality at the point: leave the germ requirement open.

Pair	Positive case	Negative mutation and required outcome
Probability uniqueness	$ q < 1$, probability profiles, shared recurrence.	$q = 1$: the contraction route must fail; distinct fixed points are possible.
Generic orbital comparison	Continuous profiles with a common base value.	Different base values: no equality conclusion.
Behavioral synthesis	Sorted output and permutation for every input.	Sorted zero-filled output of the correct length: reject the permutation claim.
Independence	A certified joint product law.	Equal marginals only: do not infer independence.
Frey-model transport	Exact division backed by divisibility.	Arbitrary integer division cast as rational division: expose the missing guard.
Jet differentiation	Input order $N + 1$ for requested output order N .	Input order N only: request more data or lower the stated guarantee.
Residual certificate	Existing root, same constant term, unit derivative.	Nonunit derivative or wrong root branch: reject the bridge.
Refinement scope	A property reused through a typed context embedding.	Sibling-branch proof with matching printed names: reject reuse.
Guard handling	A proof of the guard precedes the conditional conclusion.	Conclusion used to justify its own guard: reject cyclic assembly.
Construction	A witness and its complete requested specification.	A weaker law or finite tests substituted for the universal law: keep the request open.
Structure choice	Fixed operations and a proved transport map.	Another instance chosen to make the proof work: report a statement/structure change.

The ten executed Python tests accompanying this article cover only concrete arithmetic and finite-probability neighbors from this table. The table itself is a specification for a future frontend test suite, not a suite already passed by an implemented SCHIST elaborator.

18.4 Reader accuracy and stopping criteria

Before revealing expanded evidence, ask readers to reconstruct the actual claim: its quantifier order, region, branch, precision, completeness, and assumptions. Compare their answers with the elaborated statement. Repeat after showing the expanded contract view. Include equally plausible positive and negative neighbors, not just attractive success examples.

Set practical improvement thresholds after a pilot and before the main study. The language layer should be reduced or abandoned if its gains disappear in L2, if it increases semantic misunderstanding, or if its maintenance cost outweighs authoring savings. A successful theorem library, Leant adapter, or contract-aware document view is a legitimate outcome, not a failed language project.

19 Answers to the next-round questions

The two syntheses ask overlapping but not identical questions. The following crosswalk makes this article’s decisions explicit without pretending that unperformed measurements are settled [1, 2].

Question family	Decision or unresolved experiment
First end-to-end artifact	Property-aware binder, ordered method requirements, and ordinary Lean export, exercised by a small affine certificate and then a guarded arithmetic case. The reference core is supplied but not compiled here.
Checker soundness and reification	State a denotation theorem before relying on a Boolean checker. The small affine example makes both denotation and an explicit original-target connection visible; general reification is future work.
Where does <code>grobner</code> sit?	An existing bounded proof-producing control. Do not label it a complete decision procedure or infer a negative theorem from failure. A full current implementation audit was not performed here.
What does the workspace add?	Proof-role metadata, demand indices, source-to-evidence links, and receipts; ordinary hypotheses and instances stay in Lean’s context. Measure the added state.
Total or partial semantics?	Preserve imported Lean’s total operations. Partial-domain expressions are an explicit package with domain-sensitive equality.
Which definitions first?	Small arithmetic first; local derivative and characteristic-profile interfaces next; residual jets follow. Choose further packages by measured task coverage, not surface elegance.
How much may synthesis choose?	A candidate satisfying an explicitly delegated specification. It may not alter fixed domains, structures, branch conventions, quantifiers, or the requested law.
What stays visible?	Meaning-bearing conditions and unresolved requirements. The default display is tested through blind semantic-reading tasks.
What is Leant’s contribution?	Bounded structural composition and behavior-guided candidate search for exact property obligations. Measure separately from direct theorem lookup and from already implemented Length filtering.
How do negative outcomes work?	Keep search outcomes, modeled counterexamples, and original-target refutations distinct. Promotion requires checked logical bridges.
When is accelerated execution worthwhile?	No threshold is asserted. Measure actual certificates, audit transitive axioms, and require explicit policy. Historical timings in the syntheses do not establish a universal crossover.
How do replay and migration work?	Retain proof/certificate authority at a pin; recheck statement, interpretation, dependencies, and destination policy for an upgrade.
Can the negative catalogue be executed?	Yes as an implementation target, with positive neighbors and named rejection boundaries. Only the supplementary finite arithmetic cases were executed here.
Is another large round of prose enough?	No productivity claim follows from this article. The requested new type-system direction is specified here; the next discriminating artifact is a Lean-checked vertical slice and the L2/L3 comparison.

20 Conclusion

A more mathematical proof language should not become less precise about what its objects are, what its methods require, or what its computations establish. It should make those facts easier to express once and reuse correctly.

SCHIST places a proof-backed interface around the objects of an argument: open refinements for local properties, relational clauses for coupled objects, region-indexed observations for local mathematics, and behavioral contracts for functions and programs. The next intended step determines which parts of that interface must be established. Checked theorem applications, not informal adjectives or solver confidence, connect the requirements to the conclusion.

The proposed extension is deliberately conservative. Lean already provides the foundational expressiveness; the work is to integrate its facilities into a predictable mathematical elaboration discipline. The probability example shows why that integration must preserve weak hypotheses. The derivative example shows why it must preserve locality and induction scope. The Frey package shows both that dependent types already carry rich mathematical constraints and that some proof bodies need no new syntax at all.

The decisive experiment is consequently not whether a page of proposed syntax looks shorter. It is whether an author, using the same libraries and services, can state the right theorem, complete its proof, understand its actual guarantee, and maintain it with less effort when property-aware typing is available. That is the standard by which this proposal should be kept, narrowed, or replaced.

A Supplementary artifacts and validation

The archive contains the self-contained article source and its compiled PDF, `SchistCore.lean`, `check_examples.py`, its recorded `checks.log`, a source register, and build instructions. The Lean file is an unexecuted reference; the Python program is an executed finite sanity check, not a replacement for it.

The ten Python test methods all passed in the preparation environment. They check affine denotation on a finite family and a corrupted certificate; length-and-sortedness versus permutation; guarded rational cancellation under total and partial readings; derivative precision loss; nonunit residual and wrong-branch examples; nonuniqueness of the affine law at $q = 1$; missing normalization in the generic profile theorem; equal marginals without independence; and integral versus rational division.

The affine denotation test evaluates 50 expression fixtures at the 13 inputs $0, \dots, 12$. These 650 point checks are finite evidence only. The general soundness argument is the structural induction in Section 16, with its unexecuted formal counterpart in the Lean file. No test count is interpreted as a measure of mathematical coverage, frontend correctness, or expected productivity.

The document is compiled using strict pdfLaTeX through `latexmk`. The PDF is rendered for visual review; source, PDF, and supplementary-file hashes are recorded in `validation.json`. The build does not invoke Lean, alter any repository, or assert that the source repositories were built locally.

B Source identities and inspection scope

The register below records Git blob identities returned by the repository connector. A blob is an identity for that file's contents, not a claim that all files were read from one atomic repository snapshot. The report-B read was explicitly pinned to the Brainstorm commit stated in Section 2. Paths are given in full in `source_manifest.json`; the following short names match the bibliography.

Source	Git blob	Scope
Round-two report A	ae7c21dac1ba615e36dff07ea6f6a1c433a40108	Full main TeX
Round-two report B	de10daae20af19285ee07a476caa94e577f7d7d9	Full main TeX
Autonomous derivatives	1422b87f31073fed898ff0c8a080ea0a798aef5e	Full source
Affine independent copy	74d21739fbc57b810b64f851b08c8607332188af	Full source
Frey package	db1cdbbec434a118a917b33580fc8c81c4fff221	Full definition
FLT proof path	5a5d69b8f946ddc7587e300632cf924d6bf3373c	Full guide
FLT terminal proof	285ecf230276505ab9ddbb0ba025665b68196891	Body; preamble shape
Leant verification	caee3c51cfd226e6c2bb9887f80293343abd486c	Full module
Leant README	7d085baf3fc5069319ff515611c5802a538b5af6	Selected README
Leant internals	1723e1409060ee9eaabe21ae2ca9c37be6eaa9a7	Origin and verification
Leant Length	ab4767cd6a466583a28de56eea7080bcd44e9f3d	Opening 200 lines
Leant behavior proposal	ef0cc975a280d24b2d73df946999ab2cfb457689	Abstract and summary

The official Lean reference pages and the primary papers listed below were consulted as background sources. Their use does not imply that the new language was tested against the current Lean release. The article’s claims about existing Lean source are source-inspection claims; its claims about prior experiments are explicitly attributed to the reports that record them.

References

- [1] Codex. *Mathematical Objects, Certified Computation*. Revised round-two synthesis, Brainstorm, 6 September 2026. <https://github.com/VladimirReshetnikov/Brainstorm/blob/main/docs/round-2/synthesis/unified-report.tex>.
- [2] Claude. *Nine Workbenches: Certified Computation in a Mathematician’s Proof Language over Lean*. Revised round-two synthesis, Brainstorm. https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/unified_report/unified_report.tex.
- [3] V. Reshetnikov and contributors. ProveIt, *Iterated derivatives of a solution of an autonomous equation*. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [4] V. Reshetnikov and contributors. ProveIt, *Contractive affine independent-copy laws*. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/AffineIndependentCopy.lean>.
- [5] Contributors to the FLT development. *FreyPackage*, definition. File credits K. Buzzard, R. Van de Velde, and P. Monticone; ported from Imperial FLT. https://github.com/anthropics/fermats-last-theorem/blob/main/Definitions/Def_FLTPrelim_FreyPackage.lean.
- [6] Contributors to anthropics/fermats-last-theorem. *Repository proof-path guide*. <https://github.com/anthropics/fermats-last-theorem/blob/main/PROOF-PATH.md>.
- [7] Contributors to anthropics/fermats-last-theorem. *Frey-package terminal contradiction*, solution file. https://github.com/anthropics/fermats-last-theorem/blob/main/P2M/Sol/S_FreyPackage_no_frey_package.lean.
- [8] V. Reshetnikov and contributors. Leant, *Candidate verification*, source module. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Verification.hs>.
- [9] V. Reshetnikov and contributors. Leant, *Synthesis internals*, and selected command sections of README.md. <https://github.com/VladimirReshetnikov/Leant/blob/main/docs/synth-internals.md>.
- [10] V. Reshetnikov and contributors. Leant, *Length behavioral ranking and replay-authorized filtering*. <https://github.com/VladimirReshetnikov/Leant/blob/main/docs/length-ranking.md>.
- [11] *Precise Behavioral Code Synthesis in Leant and Djex with Z3*. Technical proposal, 15 August 2026, with later status notes. https://github.com/VladimirReshetnikov/Leant/blob/main/docs/Z3_Behavioral_Synthesis_Proposal/Z3_Behavioral_Synthesis_Proposal.tex.
- [12] Lean team. *Inductive Types*. Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.
- [13] Lean team. *Validating a Lean Proof*. Lean Language Reference. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [14] Lean team. *The mvcgen tactic: Overview*. Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/>.
- [15] Lean team. *Predicate Transformers*. Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Predicate-Transformers/>.
- [16] William Lovas and Frank Pfenning. *Refinement Types for Logical Frameworks and Their Interpretation as Proof Irrelevance*. 2010. arXiv:1009.1861. <https://arxiv.org/abs/1009.1861>.
- [17] Cezary Kaliszyk and Karol Pąk. *Semantics of Mizar as an Isabelle Object Logic*. Journal of Automated Reasoning 63 (2019), 557–595; online 2018. <https://doi.org/10.1007/s10817-018-9479-z>.
- [18] Florian Kammüller and Markus Wenzel. *Locales: A Sectioning Concept for Isabelle*. University of Cambridge Computer Laboratory, Technical Report 449, 1998. <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-449.html>.